UNIVERSITY OF WESTMINSTER

INFORMATICS INSTITUTE OF TECHNOLOGY

Informatics Institute of Technology

Client-Server Architectures

5COSC022C.2

Module Leader – Mr. Cassim Farook

Muhammed Saad Mazhar

Uow Number – W20532233

IIT Number – 20221804

Table of Contents

List of tables

1. Introduction

This report documents the comprehensive testing of the RESTful Bookstore API, developed as part of the Client-Server Architectures coursework at the University of Westminster. The testing focuses on validating the API's functionality, error handling, and adherence to RESTful design principles.

Technology Stack

The API implementation utilizes the following technologies:

- **Java 11**: Core programming language
- **JAX-RS**: Java API for RESTful Web Services specification
- **Jersey**: Implementation framework for JAX-RS
- **Maven**: Project management and dependency handling
- **JSON**: Data interchange format for request/response bodies
- **Tomcat 9**: Servlet container for deployment

Testing Environment

All tests were conducted using **Postman v10.13**, a specialized API development and testing platform that allows for:

- Creation and organization of HTTP requests
- Setting request parameters, headers, and bodies
- Examination of response status codes and body content
- Verification of error handling capabilities
- Automation of test sequences

2. API Endpoint Test Case Tables

2.1 BookResource Test Cases

2.1.1 POST /books Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
POST /books	Create a book with valid data	POST	{ "title": "The Great Gatsby", "authorId": 1, "isbn": "978-0-7432-7356-5", "publicationYear": 1925, "price": 12.99, "stock": 50 }	201 Created	{ "id": 1, "title": "The Great Gatsby", "authorId": 1, "isbn": "978-0-7432-7356-5", "publicationYear": 1925, "price": 12.99, "stock": 50 }	Pass
POST /books	Create a book with invalid author ID (not found)	POST	{ "title": "The Great Gatsby", "authorId": 999, "isbn": "978-0-7432-7356-5", "publicationYear": 1925, "price": 12.99, "stock": 50 }	404 Not Found	{ "error": "Author Not Found", "message": "Author with ID 999 does not exist." }	Pass
POST /books	Create a book with invalid publication year (future)	POST	{ "title": "Future Book", "authorId": 1, "isbn": "978-0-1234-5678-9", "publicationYear": 2030, "price": 15.99, "stock": 30 }	400 Bad Request	{ "error": "Invalid Input", "message": "Publication year cannot be in the future." }	Pass
POST /books	Create a book with empty title	POST	{ "title": "", "authorId": 1, "isbn": "978-0-1234-5678-9", "publicationYear": 2020, "price":	400 Bad Request	{ "error": "Invalid Input", "message": "Book title cannot be empty." }	Pass

			15.99, "stock": 30 }			
POST /books	Create a book with negative price	POST	{ "title": "Discount Book", "authorId": 1, "isbn": "978-0-1234-5678-9", "publicationYear" : 2020, "price": -5.99, "stock": 30 }	400 Bad Request	{ "error": "Invalid Input", "message": "Price must be greater than zero." }	Pass

Table 1 - POST /books Endpoint

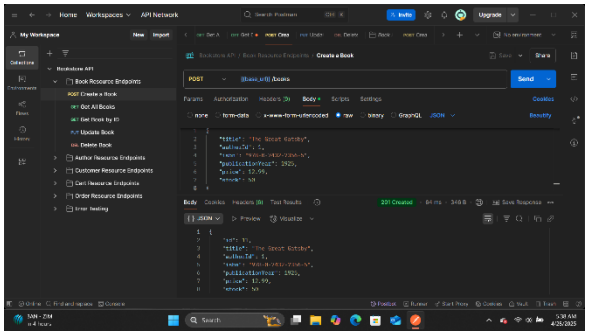

Figure 2 - Create a book with valid data

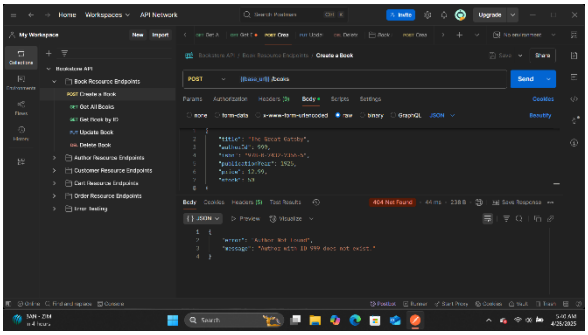

Figure 1 - Create a book with invalid author ID (not found)

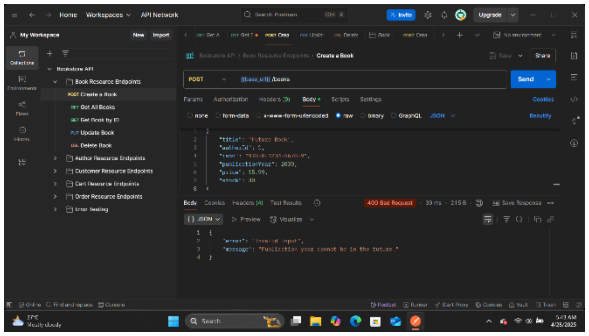

Figure 4 - Create a book with invalid publication year

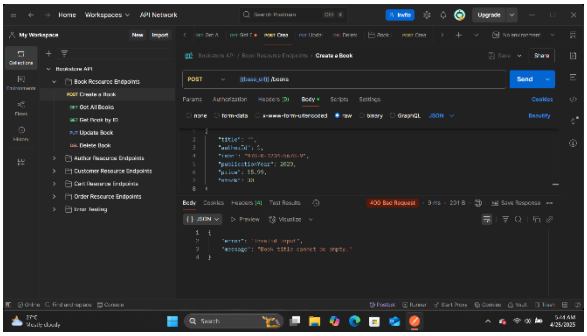

Figure 3 - Create a book with empty title

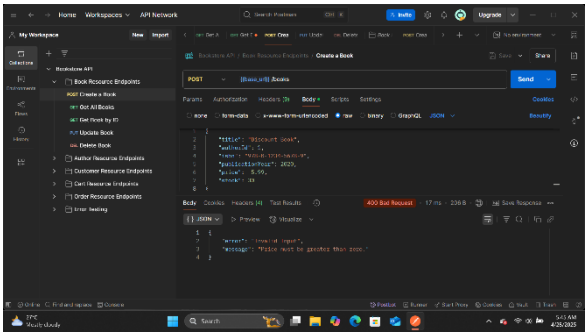

Figure 5 - Create a book with negative price

2.1.2 GET /books Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
GET /books	Get all books	GET	N/A	200 OK	[{ "id": 1, "title": "The Great Gatsby", ... }, { "id": 2, "title": "To Kill a Mockingbird", ... }]	Pass
GET /books	Get all books (empty list)	GET	N/A	200 OK	[]	Pass

Table 2 - GET /books Endpoint

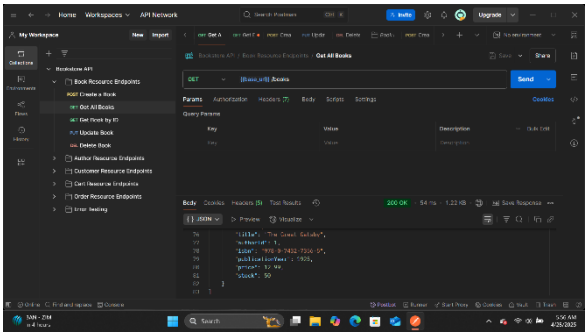

Figure 7 - Get all books

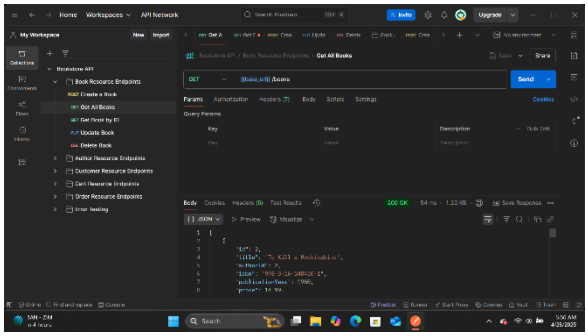

Figure 6 - Get all books (empty list)

2.1.3 GET /books/{id} Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
GET /books/1	Get book by valid ID	GET	N/A	200 OK	{ "id": 1, "title": "The Great Gatsby", "authorId": 1, "isbn": "978-0-7432-7356-5", "publicationYear": 1925, "price": 12.99, "stock": 50 }	Pass
GET /books/999	Get book by invalid ID	GET	N/A	404 Not Found	{ "error": "Book Not Found", "message": "Book with ID 999 does not exist." }	Pass

Table 3 - GET /books/{id} Endpoint

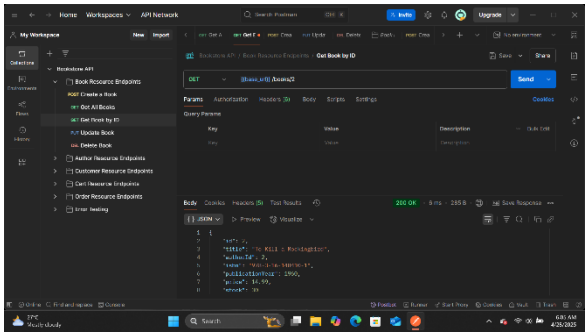

Figure 9 - Get book by valid ID

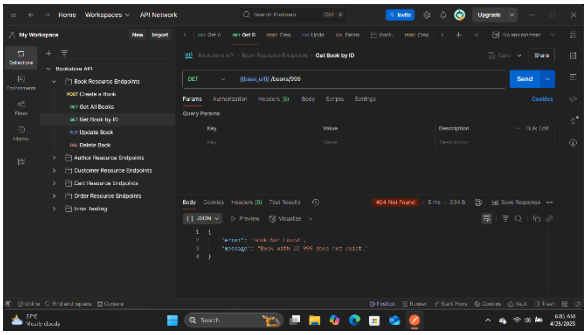

Figure 8 - Get book by invalid ID

2.1.4 PUT /books/{id} Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
PUT /books/1	Update book with valid data	PUT	{ "title": "The Great Gatsby (Updated)", "authorId": 1, "isbn": "978-0-7432-7356-5", "publicationYear": 1925, "price": 14.99, "stock": 45 }	200 OK	{ "id": 1, "title": "The Great Gatsby (Updated)", "authorId": 1, "isbn": "978-0-7432-7356-5", "publicationYear": 1925, "price": 14.99, "stock": 45 }	Pass
PUT /books/999	Update book with non-existent ID	PUT	{ "title": "Non-existent Book", "authorId": 1, "isbn": "978-0-1111-2222-3", "publicationYear": 2020, "price": 19.99, "stock": 20 }	404 Not Found	{ "error": "Book Not Found", "message": "Book with ID 999 does not exist." }	Pass
PUT /books/1	Update book with invalid data (negative stock)	PUT	{ "title": "The Great Gatsby", "authorId": 1, "isbn": "978-0-7432-7356-5", "publicationYear": 1925, "price": 14.99, "stock": -10 }	400 Bad Request	{ "error": "Invalid Input", "message": "Stock cannot be negative." }	Pass

Table 4 - PUT /books/{id} Endpoint

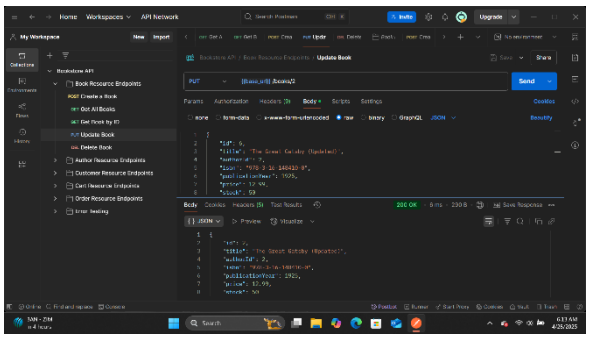

Figure 11 - Update book with valid data

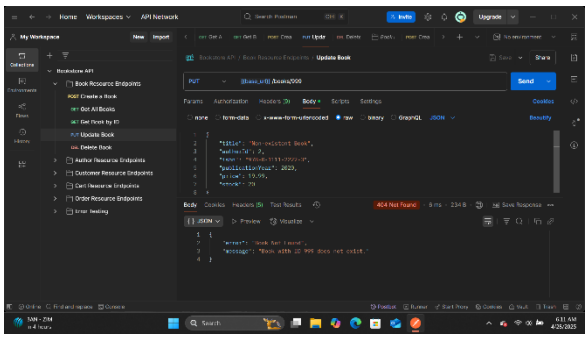

Figure 10 - Update book with non-existent ID

2.1.5 DELETE /books/{id} Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
DELETE /books/1	Delete book by valid ID	DELETE	N/A	204 No Content	N/A	Pass
DELETE /books/999	Delete book by invalid ID	DELETE	N/A	404 Not Found	{ "error": "Book Not Found", "message": "Book with ID 999 does not exist." }	Pass

Table 5 - DELETE /books/{id} Endpoint

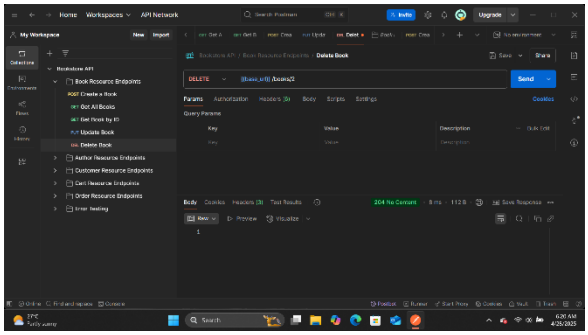

Figure 13 - Delete book by valid ID

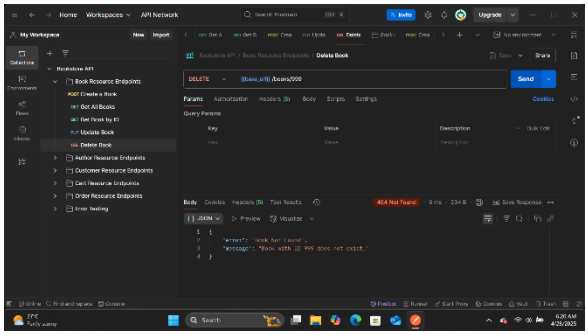

Figure 12 - Delete book by invalid ID

2.2 AuthorResource Test Cases

2.2.1 POST /authors Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
POST /authors	Create an author with valid data	POST	{ "name": "F. Scott Fitzgerald", "biography": "American novelist known for The Great Gatsby" }	201 Created	{ "id": 1, "name": "F. Scott Fitzgerald", "biography": "American novelist known for The Great Gatsby" }	Pass
POST /authors	Create an author with empty name	POST	{ "name": "", "biography": "Author biography" }	400 Bad Request	{ "error": "Invalid Input", "message": "Author name cannot be empty." }	Pass

Table 6 - POST /authors Endpoint

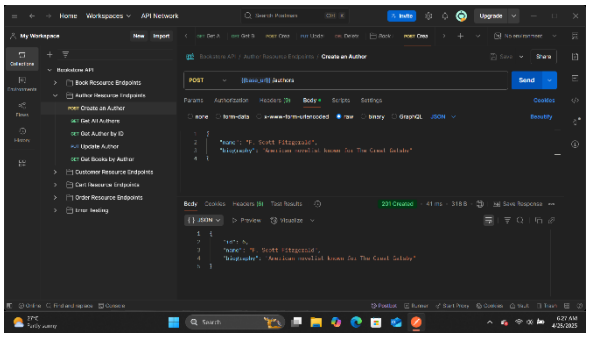

Figure 15 - Create an author with valid data

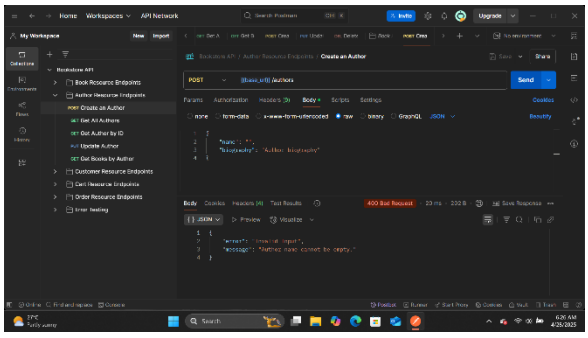

Figure 14 - Create an author with empty name

2.2.2 GET /authors Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
GET /authors	Get all authors	GET	N/A	200 OK	[{ "id": 1, "name": "F. Scott Fitzgerald", ... }, { "id": 2, "name": "Harper Lee", ... }]	Pass
GET /authors	Get all authors (empty list)	GET	N/A	200 OK	[]	Pass

Table 7 - GET /authors Endpoint

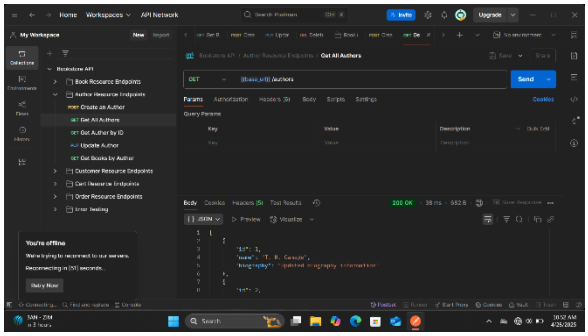

Figure 17 - Get all authors

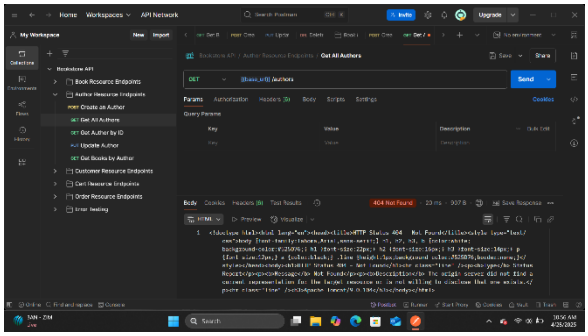

Figure 16 - Get all authors (Empty List)

2.2.3 GET /authors/{id} Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
GET /authors/1	Get author by valid ID	GET	N/A	200 OK	{ "id": 1, "name": "F. Scott Fitzgerald", "biography": "American novelist known for The Great Gatsby" }	Pass
GET /authors/999	Get author by invalid ID	GET	N/A	404 Not Found	{ "error": "Author Not Found", "message": "Author with ID 999 does not exist." }	Pass

Table 8 - GET /authors/{id} Endpoint

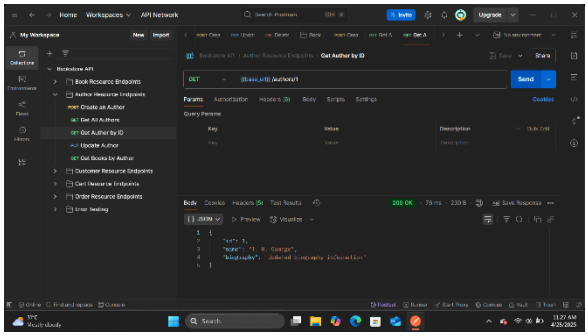

Figure 18 - Get author by valid ID

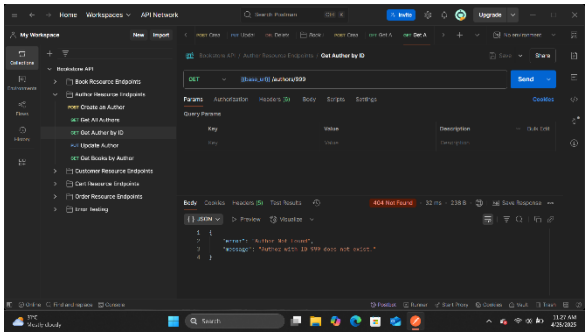

Figure 19 - Get author by invalid ID

2.2.4 PUT /authors/{id} Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
PUT /authors/1	Update author with valid data	PUT	{ "name": "F. Scott Fitzgerald", "biography": "Updated biography of the author" }	200 OK	{ "id": 1, "name": "F. Scott Fitzgerald", "biography": "Updated biography of the author" }	Pass
PUT /authors/999	Update author with non-existent ID	PUT	{ "name": "Unknown Author", "biography": "Author does not exist" }	404 Not Found	{ "error": "Author Not Found", "message": "Author with ID 999 does not exist." }	Pass
PUT /authors/1	Update author with empty name	PUT	{ "name": "", "biography": "Biography content" }	400 Bad Request	{ "error": "Invalid Input", "message": "Author name cannot be empty." }	Pass

Table 9 - PUT /authors/{id} Endpoint

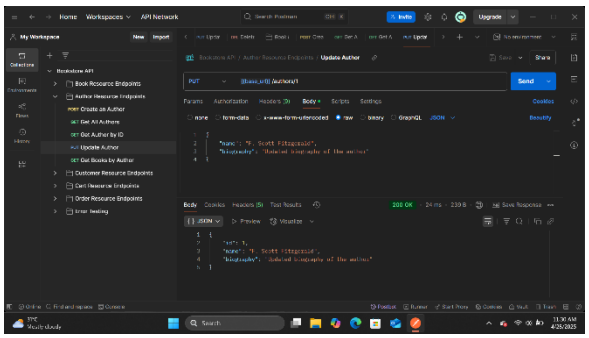

Figure 21 - Update author with valid data

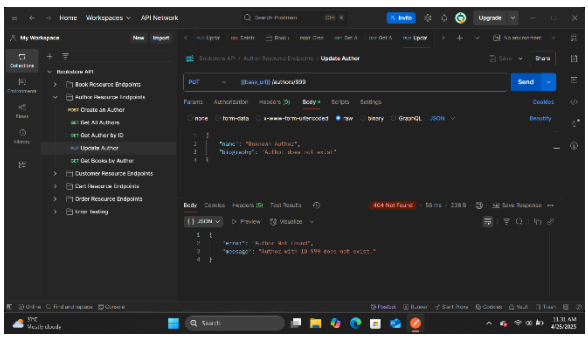

Figure 20 - Update author with non-existent ID

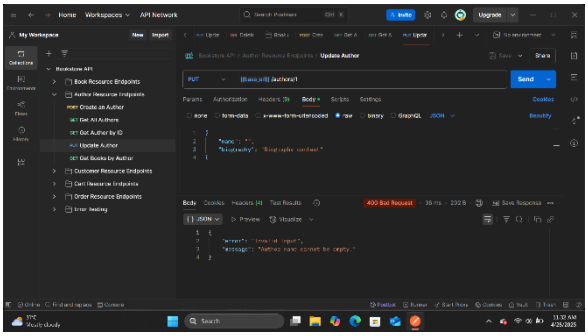

Figure 22 - Update author with empty name

2.2.5 DELETE /authors/{id} Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
DELETE /authors/2	Delete author by valid ID	DELETE	N/A	204 No Content	N/A	Pass
DELETE /authors/999	Delete author by invalid ID	DELETE	N/A	404 Not Found	{ "error": "Author Not Found", "message": "Author with ID 999 does not exist." }	Pass
DELETE /authors/1	Delete author with books	DELETE	N/A	400 Bad Request	{ "error": "Invalid Input", "message": "Cannot delete author with books. Remove books first." }	Pass

Table 10 - DELETE /authors/{id} Endpoint

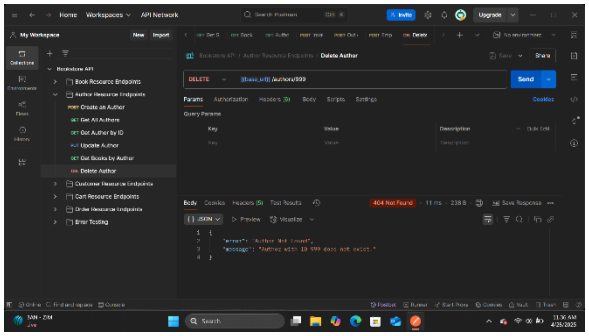

Figure 23 - Delete author by valid ID

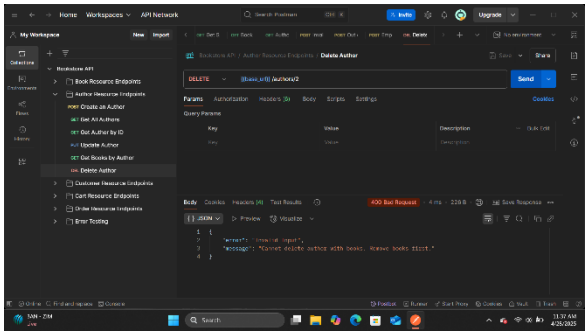

Figure 24 - Delete author by invalid ID

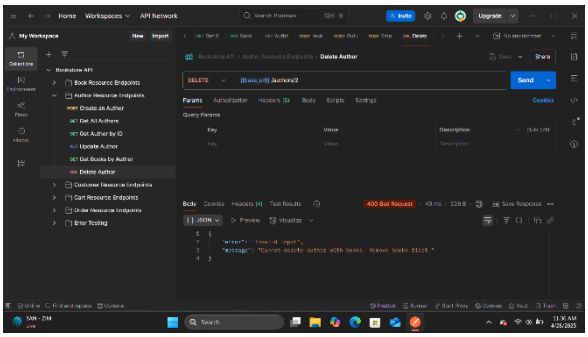

Figure 25 - Delete author with books

2.2.6 GET /authors/{id}/books Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
GET /authors/1/books	Get books by author with valid ID	GET	N/A	200 OK	[{ "id": 1, "title": "The Great Gatsby", ... }, { "id": 3, "title": "Tender Is the Night", ... }]	Pass
GET /authors/999/books	Get books by author with invalid ID	GET	N/A	404 Not Found	{ "error": "Author Not Found", "message": "Author with ID 999 does not exist." }	Pass
GET /authors/2/books	Get books by author with no books	GET	N/A	200 OK	[]	Pass

Table 11 - GET /authors/{id}/books Endpoint

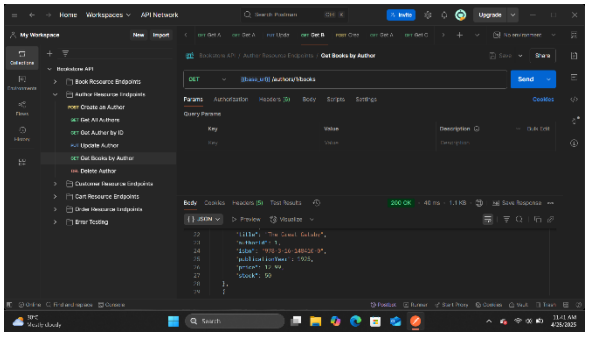

Figure 26 - Get books by author with valid ID

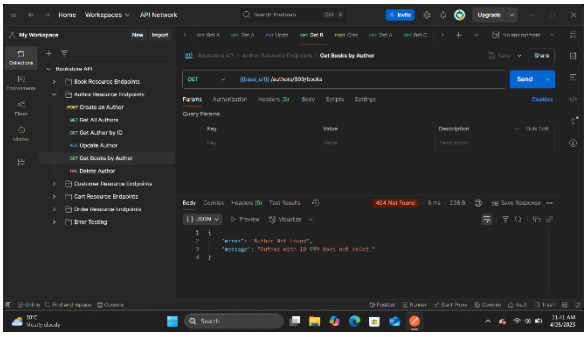

Figure 27 - Get books by author with invalid ID

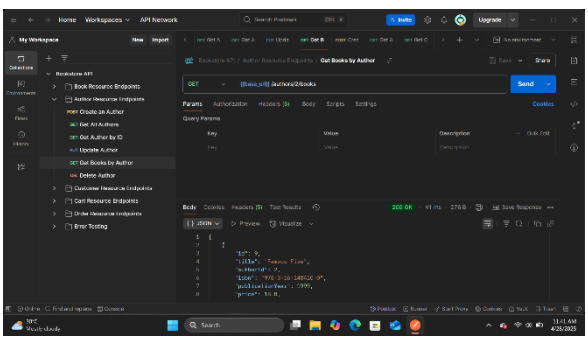

Figure 28 - Get books by author with no books

2.3 CustomerResource Test Cases

2.3.1 POST /customers Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
POST /customers	Create a customer with valid data	POST	{ "name": "John Doe", "email": "john@example.com", "password": "password123" }	201 Created	{ "id": 1, "name": "John Doe", "email": "john@example.com", "password": "password123" }	Pass
POST /customers	Create a customer with invalid email format	POST	{ "name": "John Doe", "email": "not-an-email", "password": "password123" }	400 Bad Request	{ "error": "Invalid Input", "message": "Invalid email format." }	Pass
POST /customers	Create a customer with short password	POST	{ "name": "John Doe", "email": "john@example.com", "password": "123" }	400 Bad Request	{ "error": "Invalid Input", "message": "Password must be at least 6 characters long." }	Pass
POST /customers	Create a customer with empty name	POST	{ "name": "", "email": "john@example.com", "password": "password123" }	400 Bad Request	{ "error": "Invalid Input", "message": "Customer name cannot be empty." }	Pass

Table 12 - POST /customers Endpoint

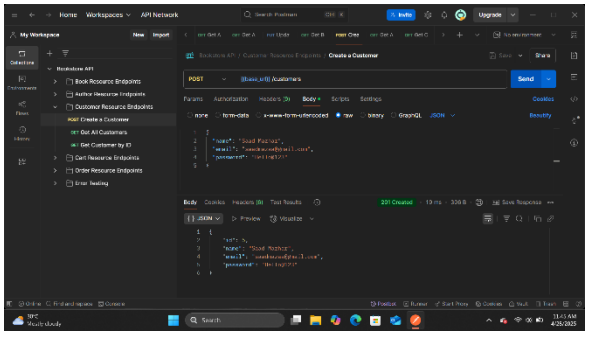

Figure 29 - Create a customer with valid data

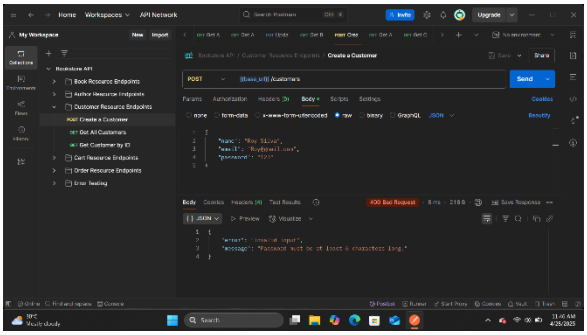

Figure 30 - Create a customer with invalid email format

2.3.2 GET /customers Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
GET /customers	Get all customers	GET	N/A	200 OK	[{ "id": 1, "name": "John Doe", "email": "john@example.com" }, { "id": 2, "name": "Jane Smith", "email": "jane@example.com" }]	Pass
GET /customers	Get all customers (empty list)	GET	N/A	200 OK	[]	Pass

Table 13 - GET /customers Endpoint

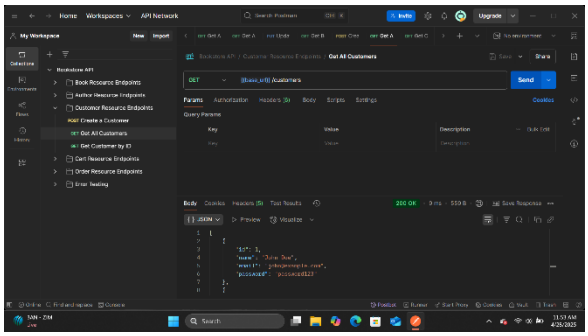

Figure 32 - Get all customers

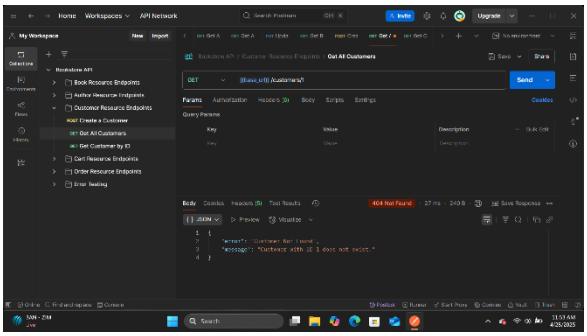

Figure 31 - Get all customers (empty list)

2.3.3 GET /customers/{id} Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
GET /customers/1	Get customer by valid ID	GET	N/A	200 OK	{ "id": 1, "name": "John Doe", "email": "john@example.com" }	Pass
GET /customers/999	Get customer by invalid ID	GET	N/A	404 Not Found	{ "error": "Customer Not Found", "message": "Customer with ID 999 does not exist." }	Pass

Table 14 - GET /customers/{id} Endpoint

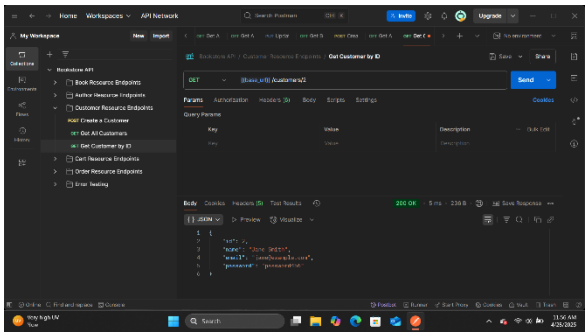

Figure 33 - Get customer by valid ID

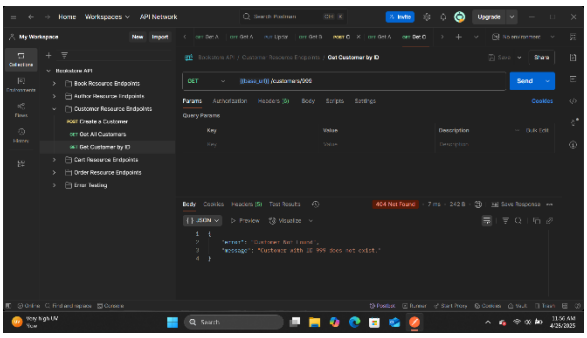

Figure 34 - Get customer by invalid ID

2.3.4 PUT /customers/{id} Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
PUT /customers/1	Update customer with valid data	PUT	{ "name": "John Doe Updated", "email": "john.updated@example.com", "password": "newpassword456" }	200 OK	{ "id": 1, "name": "John Doe Updated", "email": "john.updated@example.com", "password": "newpassword456" }	Pass
PUT /customers/999	Update customer with non-existent ID	PUT	{ "name": "Unknown Customer", "email": "unknown@example.com", "password": "password123" }	404 Not Found	{ "error": "Customer Not Found", "message": "Customer with ID 999 does not exist." }	Pass
PUT /customers/1	Update customer with invalid email	PUT	{ "name": "John Doe", "email": "invalid-email", "password": "password123" }	400 Bad Request	{ "error": "Invalid Input", "message": "Invalid email format." }	Pass

Table 15 - PUT /customers/{id} Endpoint

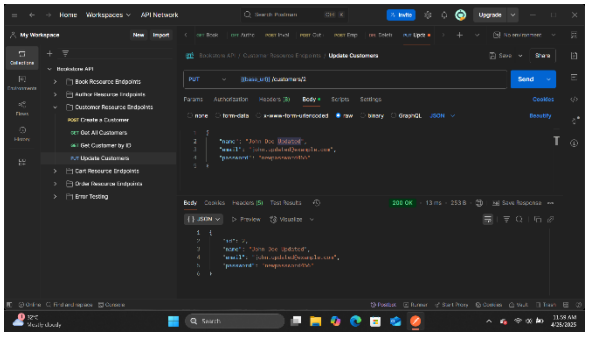

Figure 36 - Update customer with valid data

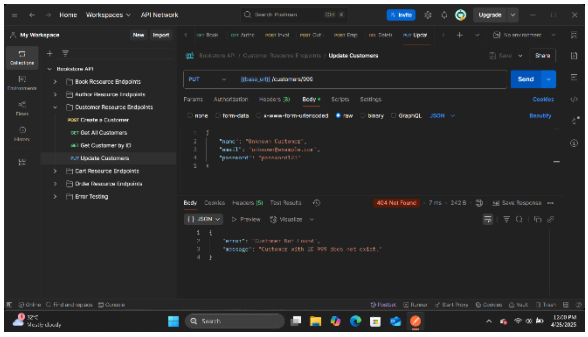

Figure 35 - Update customer with non-existent ID

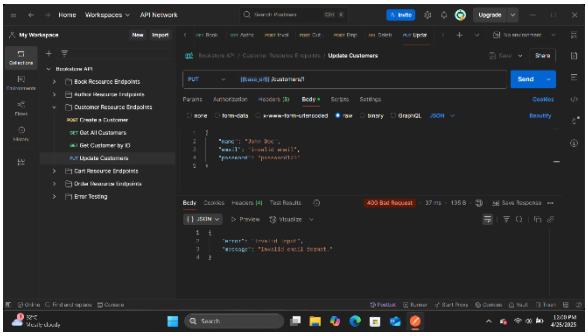

Figure 37 - Update customer with invalid email

2.3.5 DELETE /customers/{id} Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
DELETE /customers/1	Delete customer by valid ID	DELETE	N/A	204 No Content	N/A	Pass
DELETE /customers/999	Delete customer by invalid ID	DELETE	N/A	404 Not Found	{ "error": "Customer Not Found", "message": "Customer with ID 999 does not exist." }	Pass

Table 16 - DELETE /customers/{id} Endpoint

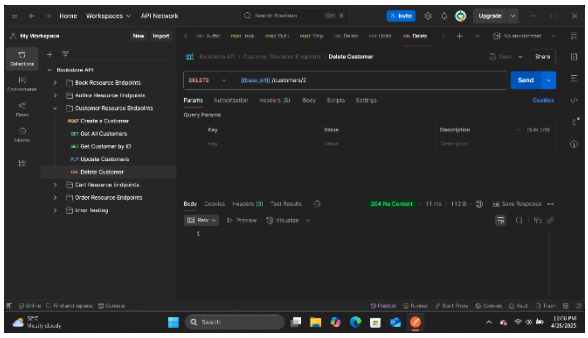

Figure 38 - Delete customer by valid ID

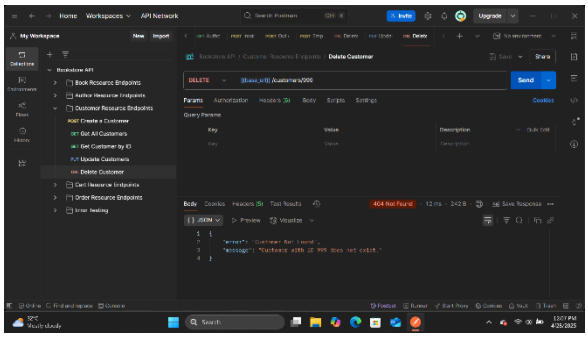

Figure 39 - Delete customer by invalid ID

2.4 CartResource Test Cases

2.4.1 GET /customers/{customerId}/cart Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
GET /customers/1/cart	Get cart for customer with valid ID	GET	N/A	200 OK	{ "customerId": 1, "items": [{ "bookId": 1, "quantity": 2 }, { "bookId": 3, "quantity": 1 }] }	Pass
GET /customers/999/cart	Get cart for customer with invalid ID	GET	N/A	404 Not Found	{ "error": "Customer Not Found", "message": "Customer with ID 999 does not exist." }	Pass
GET /customers/2/cart	Get empty cart for customer	GET	N/A	200 OK	{ "customerId": 2, "items": [] }	Pass

Table 17 - GET /customers/{customerId}/cart Endpoint

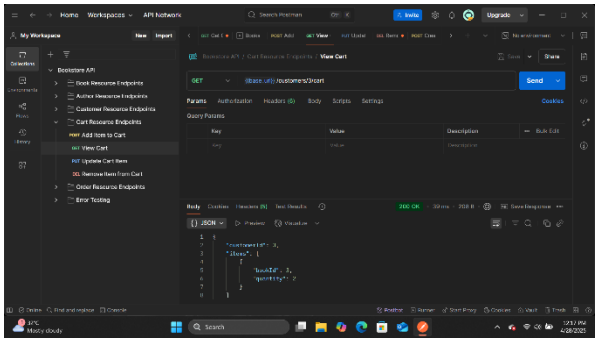

Figure 40 - Get cart for customer with valid ID

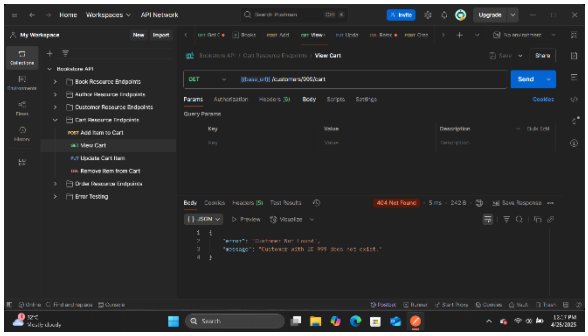

Figure 41 - Get cart for customer with invalid ID

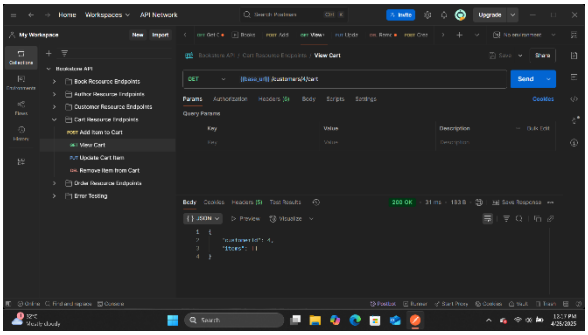

Figure 42 - Get empty cart for customer

2.4.2 POST /customers/{customerId}/cart/items Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
POST /customers/1/cart/items	Add item to cart with valid data	POST	{ "bookId": 1, "quantity": 2 }	200 OK	{ "customerId": 1, "items": [{ "bookId": 1, "quantity": 2 }] }	Pass
POST /customers/999/cart/items	Add item to cart for customer with invalid ID	POST	{ "bookId": 1, "quantity": 2 }	404 Not Found	{ "error": "Customer Not Found", "message": "Customer with ID 999 does not exist." }	Pass
POST /customers/1/cart/items	Add non-existent book to cart	POST	{ "bookId": 999, "quantity": 2 }	404 Not Found	{ "error": "Book Not Found", "message": "Book with ID 999 does not exist." }	Pass
POST /customers/1/cart/items	Add item with negative quantity	POST	{ "bookId": 1, "quantity": -1 }	400 Bad Request	{ "error": "Invalid Input", "message": "Quantity must be greater than zero." }	Pass
POST /customers/1/cart/items	Add item with quantity exceeding stock	POST	{ "bookId": 1, "quantity": 1000 }	400 Bad Request	{ "error": "Out of Stock", "message": "Not enough stock. Available: 50" }	Pass

Table 18 - POST /customers/{customerId}/cart/items Endpoint

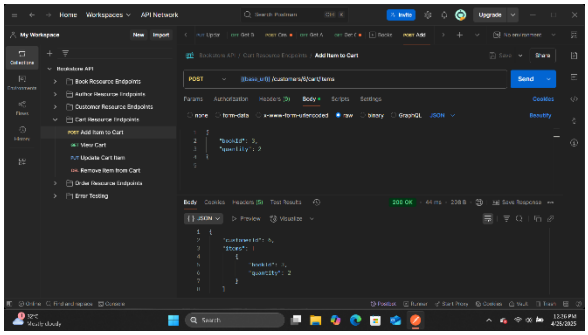

Figure 44 - Add item to cart with valid data

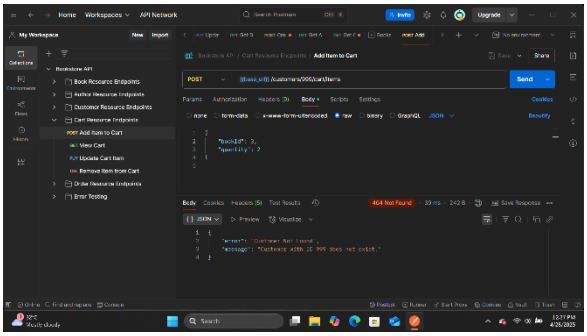

Figure 43 - Add item to cart for customer with invalid ID

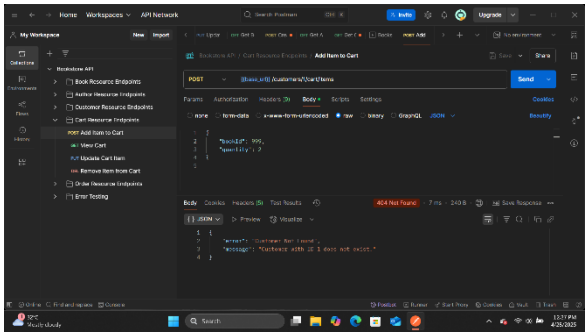

Figure 46 - Add non-existent book to cart

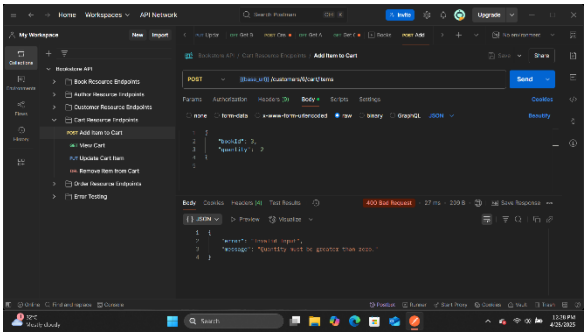

Figure 45 - Add item with negative quantity

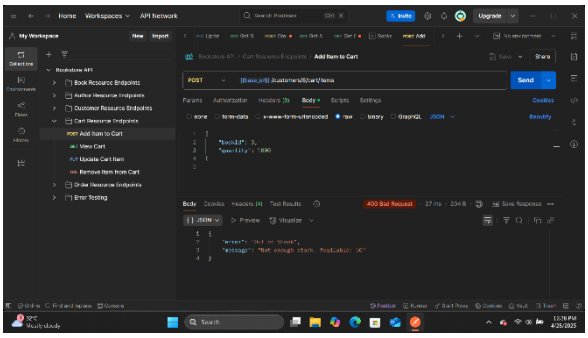

Figure 47 - Add item with quantity exceeding stock

2.4.3 PUT /customers/{customerId}/cart/items/{bookId} Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
PUT /customers/1/cart/items/1	Update cart item with valid data	PUT	{ "quantity": 3 }	200 OK	{ "customerId": 1, "items": [{ "bookId": 1, "quantity": 3 }] }	Pass
PUT /customers/999/cart/items/1	Update cart item for customer with invalid ID	PUT	{ "quantity": 3 }	404 Not Found	{ "error": "Customer Not Found", "message": "Customer with ID 999 does not exist." }	Pass
PUT /customers/1/cart/items/999	Update non-existent cart item	PUT	{ "quantity": 3 }	404 Not Found	{ "error": "Cart Not Found", "message": "Book with ID 999 not found in cart." }	Pass
PUT /customers/1/cart/items/1	Update cart item with negative quantity	PUT	{ "quantity": -1 }	400 Bad Request	{ "error": "Invalid Input", "message": "Quantity must be greater than zero." }	Pass
PUT /customers/1/cart/items/1	Update cart item with quantity exceeding stock	PUT	{ "quantity": 1000 }	400 Bad Request	{ "error": "Out of Stock", "message": "Not enough stock. Available: 50" }	Pass

Table 19 - PUT /customers/{customerId}/cart/items/{bookId} Endpoint

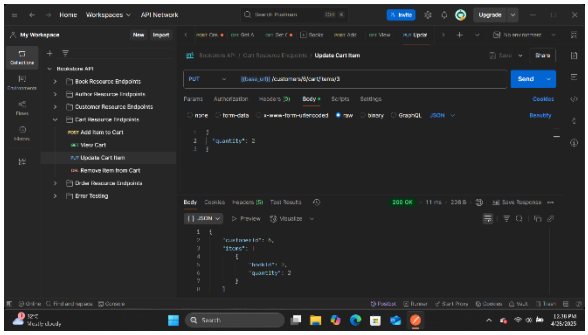

Figure 49 - Update cart item with valid data

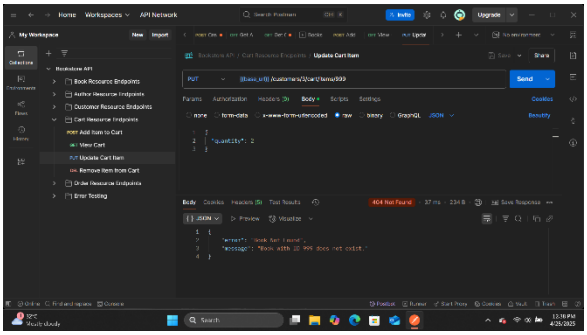

Figure 48 - Update cart item for customer with invalid ID

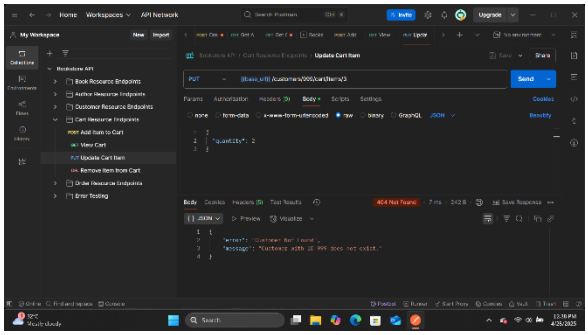

Figure 51 - Update non-existent cart item

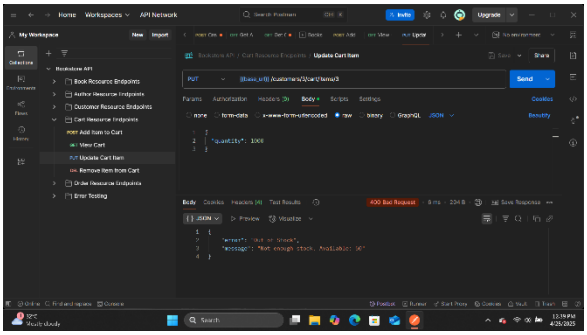

Figure 50 - Update cart item with negative quantity

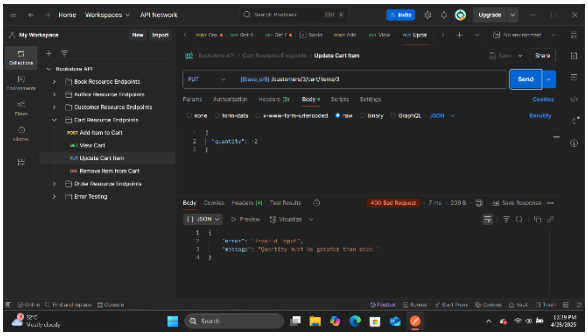

Figure 52 - Update cart item with quantity exceeding stock

2.4.4 DELETE /customers/{customerId}/cart/items/{bookId} Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
DELETE /customers/1/cart/items/1	Remove item from cart with valid IDs	DELETE	N/A	200 OK	{ "customerId": 1, "items": [] }	Pass
DELETE /customers/999/cart/items/1	Remove item from cart for customer with invalid ID	DELETE	N/A	404 Not Found	{ "error": "Customer Not Found", "message": "Customer with ID 999 does not exist." }	Pass
DELETE /customers/1/cart/items/999	Remove non-existent cart item	DELETE	N/A	404 Not Found	{ "error": "Cart Not Found", "message": "Book with ID 999 not found in cart." }	Pass

Table 20 - DELETE /customers/{customerId}/cart/items/{bookId} Endpoint

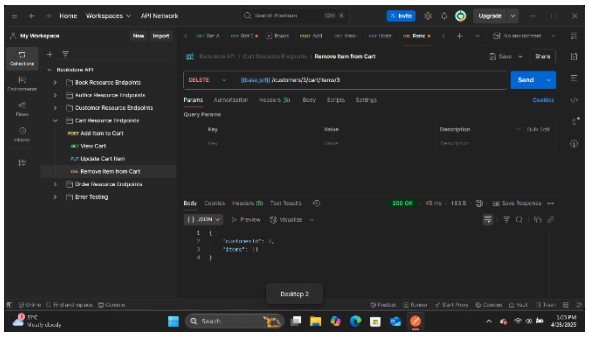

Figure 54 - Remove item from cart with valid IDs

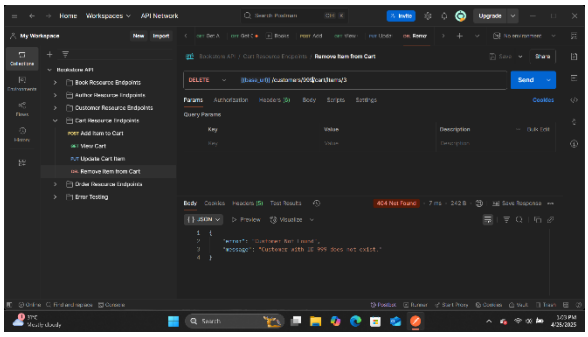

Figure 53 - Remove item from cart for customer with invalid ID

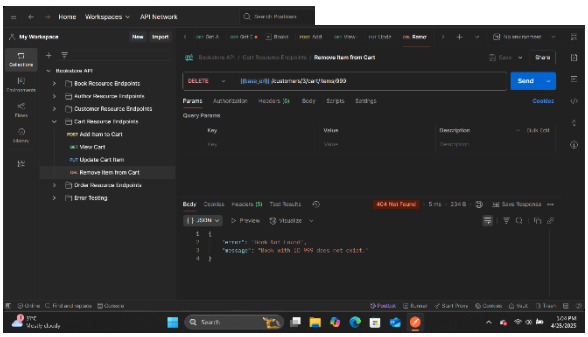

Figure 55 - Remove non-existent cart item

2.5 OrderResource Test Cases

2.5.1 POST /customers/{customerId}/orders Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
POST /customers/1/orders	Create order with items in cart	POST	N/A	201 Created	{ "id": 1, "customerId": 1, "orderDate": "2025-04-27T12:00:00", "totalAmount": 25.98, "items": [{ "bookId": 1, "quantity": 2, "price": 12.99 }] }	Pass
POST /customers/999/orders	Create order for customer with invalid ID	POST	N/A	404 Not Found	{ "error": "Customer Not Found", "message": "Customer with ID 999 does not exist." }	Pass
POST /customers/2/orders	Create order with empty cart	POST	N/A	400 Bad Request	{ "error": "Invalid Input", "message": "Cannot create order with empty cart." }	Pass
POST /customers/1/orders	Create order with item exceeding stock	POST	N/A	400 Bad Request	{ "error": "Out of Stock", "message": "Not enough stock for book: The Great Gatsby. Available: 5" }	Pass

Table 21 - POST /customers/{customerId}/orders Endpoint

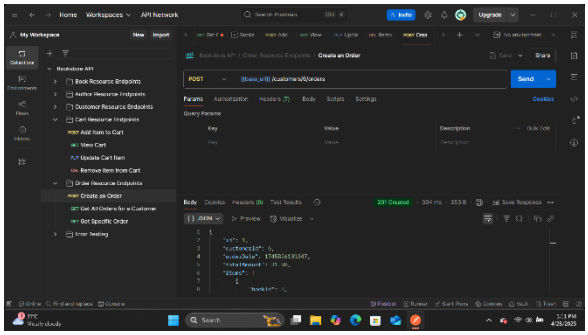

Figure 56 - Create order with items in cart

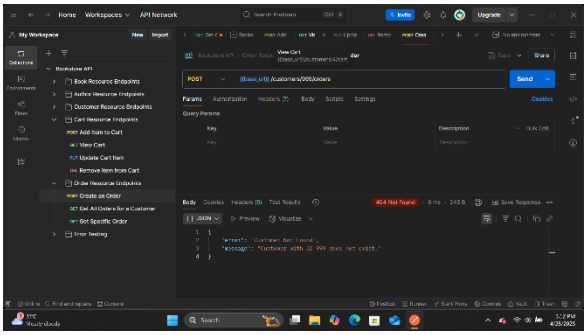

Figure 57 - Create order for customer with invalid ID

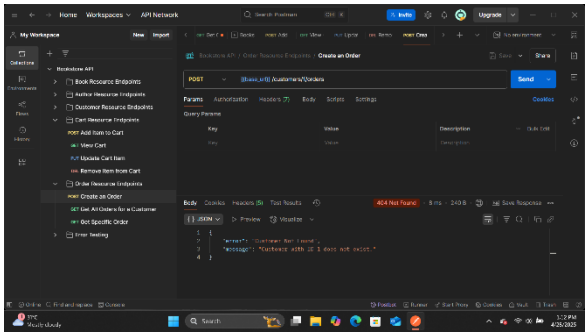

Figure 59 - Create order with empty cart

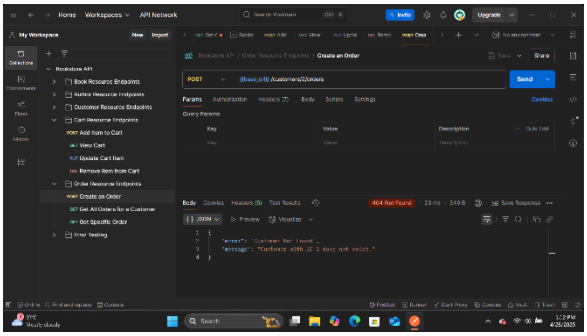

Figure 58 - Create order with item exceeding stock

2.5.2 GET /customers/{customerId}/orders Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
GET /customers/1/orders	Get orders for customer with valid ID	GET	N/A	200 OK	[{ "id": 1, "customerId": 1, "orderDate": "2025-04-27T12:00:00", "totalAmount": 25.98, "items": [{ "bookId": 1, "quantity": 2, "price": 12.99 }] }]	Pass
GET /customers/999/orders	Get orders for customer with invalid ID	GET	N/A	404 Not Found	{ "error": "Customer Not Found", "message": "Customer with ID 999 does not exist." }	Pass
GET /customers/2/orders	Get orders for customer with no orders	GET	N/A	200 OK	[]	Pass

Table 22 - GET /customers/{customerId}/orders Endpoint

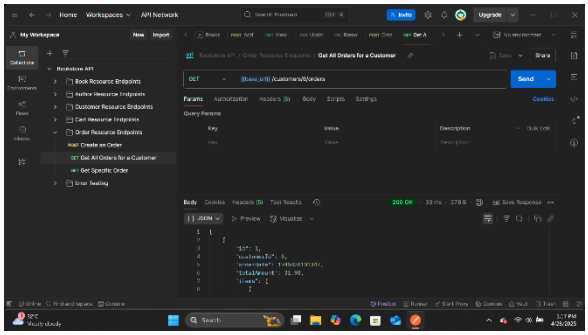

Figure 61 - Get orders for customer with valid ID

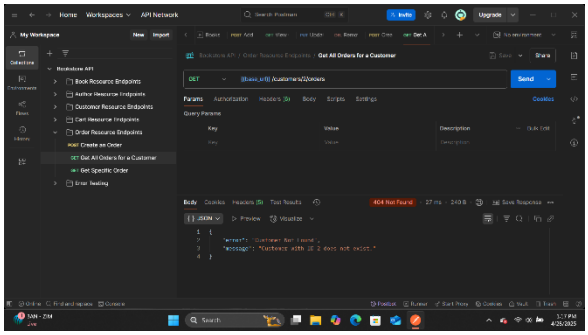

Figure 60 - Get orders for customer with invalid ID

2.5.3 GET /customers/{customerId}/orders/{orderId} Endpoint:

Endpoint	Description	HTTP Method	Request Body (JSON)	Expected HTTP Status Code	Expected Response Body (JSON)	Actual Result
GET /customers/1/orders/1	Get order with valid customer and order IDs	GET	N/A	200 OK	{ "id": 1, "customerId": 1, "orderDate": "2025-04-27T12:00:00", "totalAmount": 25.98, "items": [{ "bookId": 1, "quantity": 2, "price": 12.99 }] }	Pass
GET /customers/999/orders/1	Get order for customer with invalid ID	GET	N/A	404 Not Found	{ "error": "Customer Not Found", "message": "Customer with ID 999 does not exist." }	Pass
GET /customers/1/orders/999	Get non-existent order	GET	N/A	404 Not Found	{ "error": "Customer Not Found", "message": "Order with ID 999 not found for customer 1" }	Pass

Table 23 - GET /customers/{customerId}/orders/{orderId} Endpoint

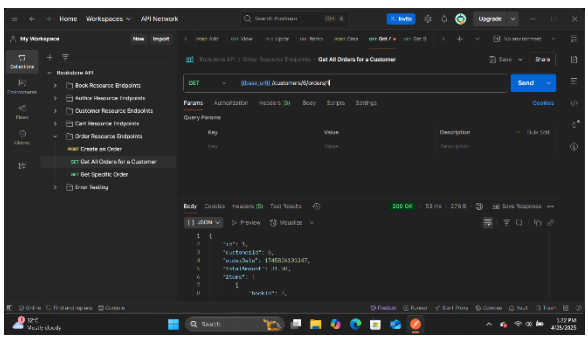

Figure 63 - Get order with valid customer and order IDs

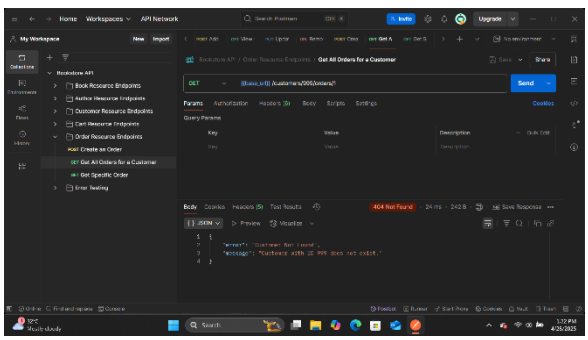

Figure 62 - Get order for customer with invalid ID